

퀄리저널 관리자 시스템 개발 파일 분석 및 수정 제안

개요

이 보고서는 **QualiJournal 관리자 시스템**을 운영하는 과정에서 발생한 문제점과, 이를 개선하기 위해 **어떤 파일을 점검·수정해야 하는지**를 정리한 자료입니다. 이전 보고서에서 지적된 문제점(토큰 인증 문제, UI 불안정, 새로고침 불일치, 동적 게이트 설정, 요약/보고서 기능 오류 등)을 바탕으로, **프로젝트 구조**와 **주요 파일**을 분석하고 구체적인 수정 방향을 제안합니다.

1. 프로젝트 구조

- **admin/** - 관리자 페이지 클라이언트와 서버 API가 포함된 디렉터리. 주로 HTML/JS(UI)와 FastAPI 서버 코드가 위치한다.
- **archive/** - 생성된 보고서와 번역/요약 파일 저장소. 동적으로 생성되는 결과물(예: 보고서 MD, 요약 MD, CSV)이 쌓인다.
- **data/, feeds/** - 뉴스 수집에 사용되는 소스 및 편집 규칙(JSON). `feeds/community_sources.json`, `feeds/official_sources.json`, `feeds/editor_rules.json`, `feeds/keyword_synonyms.json` 등.
- **tools/** - 수집·분류·요약·선정·발행 작업을 하는 스크립트들이 모여 있다. `approve_top20.py`, `enrich_cards.py`, `make_daily_report.py` 등.
- **engine_core.py** - 기사 점수화 및 랭킹 알고리즘. 보충/재구성 단계에서 사용된다.
- **orchestrator.py** - 키워드 수집, 승인(선정), 발행, 요약 등을 관리하는 파이프라인 스크립트. `python orchestrator.py --collect-keyword <kw>` 형태로 호출된다.
- **server_quali.py** - FastAPI 서버로, 관리자 페이지에 노출되는 모든 `/api/*` 엔드포인트를 정의하고 `TaskManager`를 통해 위 스크립트를 비동기로 실행한다.
- **config.json, config_gate_example.json** - 게이트(승인 필요한 기사 수) 등 관리자 페이지에서 사용되는 설정값.

프로젝트의 핵심 흐름은 다음과 같다: 키워드를 입력하면 `orchestrator.py`가 여러 소스(`community/official`)에서 기사를 수집하고 `selected_keyword_articles.json`에 저장한다. `server_quali.py`의 API를 통해 KPI(선정/승인/Ready 수 등)를 확인하고 게이트를 설정한다. 필요한 경우 `enrich_cards.py`, `make_daily_report.py` 등의 툴을 호출해 요약이나 보고서를 생성하며, 최종적으로 `archive/` 디렉터리에 결과물을 저장하고 `admin/index.html` UI를 통해 내보낼 수 있다.

2. 주요 파일 점검 및 수정 필요 사항

2.1 `admin/index.html` - 관리자 UI

문제점

- **토큰 입력 후 미인증 상태 지속:** `saveAdminTokenJJ()`는 토큰을 `localStorage`에 저장하고 `/api/status` 호출로 유효성을 검사하는데, `updateAuthUI()` 호출 순서가 적절하지 않아 UI 배지가 갱신되지 않는 문제가 있었다. 코드 위치는 332~336라인 ¹.

- **새로고침/게이트 조절 버튼이 동작하지 않음:** DOMContentLoaded 이벤트에서 btnRefresh 와 btnKpiRefresh 에 refresh() 를 연결하고 초기 refresh() 를 실행해야 하는데, 이 코드가 빌드에 포함되지 않거나 구버전 HTML이 배포되면서 버튼이 작동하지 않았다 2 .
- **게이트 슬라이더와 API 동기화 오류:** 게이트 값을 바꾸면 updateGate() 를 통해 /api/config/gate_required 로 PATCH해야 하지만, 슬라이더 값과 실제 API 값이 불일치하는 경우가 있었다 3 .
- **복수 키워드 지원·작업 로그 가시성 부족:** 현재 UI는 단일 키워드 흐름에 최적화되어 있으며, 로그 출력 위치도 작아 운영자가 현재 진행 상태를 보기 어렵다.

수정 제안

1. **토큰 저장 및 UI 갱신 개선**
2. saveAdminTokenJJ() 에서 updateAuthUI() 호출을 토큰 저장 직후로 이동하여 저장하면 즉시 배지 색상이 바뀌도록 수정 1 .
3. /api/status 호출 실패 시 사용자에게 알림을 띄워 토큰 오류를 명확히 안내하도록 testToken() 함수 개선 4 .
4. **새로고침 버튼 이벤트 확실히 연결**
5. DOMContentLoaded 이벤트에서 refresh() 호출 및 버튼 바인딩 코드가 실행되는지 빌드 파이프라인을 점검하고, 누락 시 추가 2 .
6. btnTokenClear 가 페이지 리로드 없이 localStorage 값을 삭제하도록 clearAdminTokenJJ() 를 수정.
7. **게이트 슬라이더 동기화**
8. 슬라이더 변경 시 updateGate() 를 호출해 서버에 바로 반영하되, 성공 응답을 받은 후 UI에 반영하도록 콜백 처리 3 .
9. 게이트 값 변경 후 refresh() 를 자동 호출해 KPI와 게이트 통과 여부를 업데이트.
10. **운영자 편의 UI 개선**
11. 하단 비동기 작업 로그 영역을 확장하고, 각 작업 단계별 진행률을 보여주는 프로그레스 바 추가.
12. 여러 키워드를 순차 실행할 수 있는 큐 UI(리스트 형태) 도입.
13. KPI 영역과 Ready 영역에서 아이콘 색상으로 상태(선정/승인/거절)를 직관적으로 표시.

2.2 server_quali.py – FastAPI 백엔드

server_quali.py 는 관리자 UI와 통신하며, 기사 수집·승인·발행·보고서/요약 생성 등 주요 기능을 호출한다. 패치 파일에서는 동적 게이트 설정과 비동기 작업 관리를 추가했다 5 6 .

문제점

- **토큰 검증 실패 시 응답 일관성 부족:** authorize() 함수는 헤더 X-Admin-Token 또는 Authorization: Bearer 와 환경 변수 ADMIN_TOKEN 을 비교하지만, 오류 메시지가 HTML일 때도 있어 프론트엔드에서 처리하기 어렵다.
- **동적 게이트와 상태 값 반영 미비:** /api/status 는 gate_required 와 state_counts 를 반환하지만, ready 상태 계산이 정확하지 않아 KPI가 0으로 표시되는 경우가 있었다. 또한 gate_pass 가 True 라도 UI에 반영되지 않음 5 .
- **비동기 작업 관리:** TaskManager 가 스레드를 생성해 orchestrator.py 스크립트를 실행하지만, 동일 키워드로 여러 작업이 중복 실행되거나 오류 로그가 소실되는 문제가 있다.
- **예외 처리 및 오류 로깅 부족:** 기사 수집 중 네트워크 오류나 API 실패가 발생해도 백엔드가 500 오류만 반환하며 원인 파악이 어렵다.

수정 제안

1. **토큰 인증 로직 개선**

2. `authorize()` 에서 토큰 검증 실패 시 JSON 형태 `{ "detail": "invalid admin token" }` 를 일관되게 반환하고, 상태 코드는 401로 유지. 프론트엔드가 이를 파싱해 사용자에게 명확히 알림.
3. `ADMIN_TOKEN` 이 설정되지 않은 경우엔 토큰 검증을 건너뛰거나 경고 로그를 남기도록 옵션화.
4. **상태 계산 로직 정비**
5. `/api/status` 에서 `selected_keyword_articles.json` 과 `selected_articles.json` 파일을 제대로 읽고, `state_counts` 의 `candidate`, `ready`, `rejected` 를 정확히 계산하도록 `calc_status()` 함수를 분리하여 테스트 가능하게 작성.
6. `gate_pass` 가 변경되면 `gate_required` 값을 응답에 포함하고, UI에서 즉시 반영될 수 있도록 설명 메시지를 추가.
7. **비동기 작업 큐 개선**
8. `TaskManager` 가 실행 중인 작업 리스트를 저장하고, 동일 키워드/작업 중복 요청 시 이전 작업을 중단하거나 큐에 넣도록 로직 수정.
9. 각 작업 시작·종료 시 로그를 파일에 남기고, `/api/tasks` 엔드포인트를 통해 현재 작업 상태를 반환하도록 개선.
10. Python `subprocess` 대신 `asyncio.create_subprocess_shell` 를 사용해 비동기 실행과 스트림 반환을 안정화.
11. **예외 처리·모니터링 강화**
12. 외부 API 호출(뉴스 크롤러, 요약 API 등)의 예외를 `try/except` 로 잡아 상세 원인을 로그로 남기고, 프론트엔드에는 "수집 실패" 등 사용자 친화적인 메시지와 재시도 방법 안내.
13. Sentry 또는 Cloud Logging 연동으로 운영자에게 오류 알림을 전송.

2.3 orchestrator.py - 기사 파이프라인

`orchestrator.py` 는 키워드 수집→재구성→선정→발행 및 백업까지 전 과정을 수행한다. 현재는 명령행 인자로 `--collect-keyword`, `--approve-keyword`, `--publish-keyword` 등을 받아 순차적으로 실행한다.

문제점

- **파이프라인 간 데이터 전달 방식:** 각 단계가 공용 파일(JSON)에 의존해 읽고 쓰는데, 단계별 실패시 롤백이나 재시도가 어렵다. 예를 들어 `collect_keyword` 가 실패해도 `approve_keyword` 가 실행될 수 있다.
- **동일 키워드 중복 실행:** 백그라운드 작업으로 실행할 때 동일 키워드로 여러 스레드가 동시에 실행돼 중복 작업이 발생할 수 있다.
- **로깅-에러 처리 미흡:** 작업 실패 원인이 CLI 출력에만 남고 `server_quali.py` 에서 제대로 전달되지 않는다.

수정 제안

1. **단계별 트랜잭션 관리**
2. 각 단계 결과를 파일뿐 아니라 상태 DB(DB 파일이나 Redis 등)에 저장하고, 다음 단계 실행 전 이전 단계 성공 여부를 확인. 실패 시 재시도 하거나 사용자에게 안내.
3. `--collect-approve-publish-keyword` 와 같이 전체 파이프라인을 하나의 명령으로 실행하여 내부적으로 순차 실행하고 오류 발생 시 중단하도록 변경.
4. **중복 실행 방지**
5. PID 파일 또는 작업 레지스트리를 활용해 동일 키워드 작업이 실행 중이면 새 작업을 거부하거나 큐에 넣는다.
6. **로깅 강화**
7. Python logging 모듈을 사용해 단계별 로그를 파일과 표준 출력에 기록하고, `TaskManager` 가 이를 읽어 프론트엔드에 스트리밍하도록 수정.

2.4 engine_core.py - 기사 점수화/재구성

엔진 코드는 기사 제목·본문을 벡터화해 유사도 계산 및 중요도 점수화를 수행한다. 최근 위협 또는 편향을 줄이기 위해 d3_regression_check.py 등 도구를 제공한다.

수정 제안

- 최신 뉴스 트렌드를 반영하도록 feeds/keyword_synonyms.json 을 확장하고, engine_core.py 의 랭킹 가중치 파라미터를 config.json 에서 불러오도록 수정.
- classify_and_rank.py 등 보조 스크립트와 engine_core.py 간의 중복 로직을 통합하여 유지보수 용이성을 높임.

2.5 설정 및 데이터 파일

config.json / config_gate_example.json

- 게이트 요구 건수를 gate_required 로 정의하며, Patch 적용 후에는 /api/config/gate_required 로 수정 가능.
- require_editor_approval 등 새 필드를 추가해 관리자 승인 여부를 설정할 수 있도록 확장.

feeds 디렉터리

- community_sources.json, official_sources.json : 뉴스 RSS/사이트 리스트. 수집 품질 향상을 위해 출처를 최신화하고, 중복·편향 소스를 제거.
- editor_rules.json : 특정 키워드에 대한 편집 규칙과 블랙리스트/화이트리스트를 정의. 새 키워드에 대한 규칙을 추가하고, 키워드 간 중복 규칙을 통합.
- keyword_synonyms.json : 키워드별 유의어 모음. AI 와 인공지능 등 다양한 표현을 포함시키고 업데이트 주기 정의.

selected_articles.json / selected_keyword_articles.json

- status API에서 KPI 계산 시 읽는 핵심 파일. 스키마를 문서화하고, 백엔드에서 이 파일들을 읽을 때 예외 처리를 강화.

Tools (tools/*.py)

- 비즈니스 로직을 스크립트 형태로 분산해 두었지만 중복과 함수화 부족으로 유지보수가 어렵다. 각 스크립트를 모듈화하여 orchestrator 에서 import 형태로 호출하도록 리팩터링.
- 중요 스크립트: approve_top20.py (상위 20개 선정), merge_and_approve_top30.py, make_daily_report.py, enrich_cards.py. 이들의 파라미터와 동작 방식을 통일하고, 테스트 코드를 추가.

3. 종합적인 개선 방향

1. UI와 API 싱크 보장: 프론트엔드 index.html 에서 이벤트 바인딩과 토큰 관리 로직을 확실히 동작하도록 수정하고, 빌드/배포 시 항상 최신 HTML이 포함되도록 .gcloudignore 와 Dockerfile 설정을 확인한다.
2. 안정적인 비동기 작업 관리: server_quali.py 의 TaskManager 를 개선해 중복 실행을 방지하고, 상태 조회 API(/api/tasks)를 제공한다. 작업 로그를 파일로 저장하여 문제 발생 시 추적 가능하게 한다.

3. **동적 게이트 및 승인 정책 지원:** `config.json` 에 `gate_required`, `require_editor_approval` 등을 추가하고, `updateGate()` 와 `/api/config/gate_required` 를 통해 실시간 조정 가능하게 한다. 프론트엔드에서 변경이 즉시 KPI에 반영되도록 `refresh()` 로직을 수정한다.
4. **모듈화 및 코드 품질 향상:** 스크립트들을 라이브러리 형태로 모듈화하고, 공통 기능을 `common_utils.py` 나 `engine_core.py` 에 통합한다. 테스트 코드와 문서화를 추가해 유지보수성을 높인다.
5. **모니터링 및 알림 강화:** Cloud Logging/Sentry 연동으로 오류 발생 시 운영자에게 알림을 보내고, Slack 통합을 통해 보고서 생성·요약 결과를 자동 전달하도록 한다.
6. **데이터 소스 업데이트:** `feeds` 와 `editor_rules` 를 최신화하고, 키워드 유의어 사전을 확대하여 수집 커버리지를 높인다.

4. 결론

QualiJournal 관리자 시스템은 강력한 기능을 제공하지만, **토큰 인증 및 UI 불안정, 게이트 설정 미스매치, 비동기 작업 관리** 등의 문제로 운영자가 사용하기 어렵다는 점이 확인되었습니다. 위에서 제시한 수정 제안을 통해 UI와 백엔드 간 동기화를 강화하고, 파이프라인을 안정화하며, 설정 파일과 도구들을 통합하면 운영 편의성과 시스템 안정성을 크게 개선할 수 있습니다. 앞으로는 **코드 모듈화, 예외 처리 강화, 모니터링 도입**을 통해 지속적으로 품질을 향상하는 것이 중요합니다.

1 2 3 4 index.html

<https://github.com/cdmacs1003-cyber/quali-journal/blob/66de42e6438fcc11deda20db2cd394f285976433/admin/index.html>

5 6 patch.diff

<https://github.com/cdmacs1003-cyber/quali-journal/blob/66de42e6438fcc11deda20db2cd394f285976433/admin/patch.diff>